

PackIT AI - Technical Status Report

Database and real-time API requirements, current progress, and the work left before this is production-ready

Date 28 September 2026 | Branch **main @ d11c52a** | Flutter (Android) client + FastAPI backend

Every figure below was measured or read directly out of the working tree on the date above, not estimated from design documents. Where something does not work, the evidence is named so it can be re-checked.

1. Where the project stands today

The product is a Flutter client (13 screens) on a FastAPI backend (12 routers, 16 service modules). Two parts are genuinely engineered and defensible; the rest is a convincing demo layer that degrades to bundled sample data whenever the backend is not reachable.

Component	Status	Evidence
3D Mockup Studio (customizer)	Working - real	5 bundled .glb models scoped by product family (chips vs dairy), artwork stamped as a texture, H/V/scale/rotate. model-viewer is bundled locally, so the viewer runs with no network at all.
Packaging stack solver	Working - real	1,120 candidate laminates scored with the same permeation and Arrhenius engine; 3 tiers with meets_target, limiting_factor, predicted life, thickness, INR per 1,000 and why[] reasoning.
Physics / LCA core	Working - real	Series permeability 1/TR = sum(d/P), quality-decay ODE via scipy.solve_ivp, cost and carbon from Indian market rates. Covered by 48 backend tests.
Wizard to API contract	Working - real	The user's own moisture, fat, pH, storage temperature, humidity, shelf life and budget now reach the backend. Budget used to be hardcoded (40.0) and the shelf-life key was read wrong, so real input was silently ignored.
Recommendation screen	Working - real	Invented values removed: no 95% MATCH badge, no 70-90 confidence score, no matchScore / protection literals. It now renders solver output.
Digital Twin Simulator	Partial	The animated curves are a local heuristic (a single exponential decay); a panel underneath runs the real backend engine and admits 'backend unreachable' when it cannot.
Packaging Auditor	Partial	Camera capture and the vision proxy are real, but no packaging-defect model exists - the router's own docstring says general vision models are not trained on packaging-defect datasets.
AI Chat Assistant	Partial	Real LLM only when OPENAI_API_KEY is set; otherwise a curated canned answer, so the 'AI' label overstates what the user is reading.
Dashboard / History / Suppliers	Partial	Live when the backend answers, sample data otherwise. The UI does show an explicit 'live / sample data' badge rather than pretending.
Accounts / authentication	Missing	REQUIRE_AUTH defaults to false, no user model, every endpoint open.

Verified checks

Check	Result
flutter analyze	No issues found (clean; two pre-existing warnings were also fixed)
flutter test	2 widget tests pass
backend pytest	48 tests pass
Release APK	Builds and installs; 54.6 MB (57,270,145 bytes). The committed APK predates the solver changes.
API latency (in-process, includes the 1,120-candidate search)	/products 35 ms /recommendations/generate 276 ms /simulate/digital-twin 17 ms /simulate/shelf-life 15 ms /lca/calculate 11 ms /chat/message 11 ms (no LLM) /dashboard/summary 8 ms

2. What is not ready, and the evidence for it

- Backend unreachable from a real phone (critical). api_client.dart resolves Android to 10.0.2.2:8000, the emulator's alias for the host machine. On a physical device that address points at the phone itself, so the installed APK falls back to sample data everywhere except the 3D studio.
- No persistence (critical). STORAGE_BACKEND=memory by default: all eight collections sit in an in-process dictionary and vanish on restart. The Firestore path exists but is unconfigured.
- No authentication (critical). Every endpoint is open; anyone who can reach the host can read and write all data.
- Release build signed with the debug keystore. build.gradle.kts still carries the 'TODO: add your own signing config' comment. Such an APK cannot be published, and each upgrade forces users to uninstall first.
- Committed APK is stale. Built 09:24; the solver recommendations landed 09:42-10:09. The code on main is ahead of the shipped binary.
- Tier rationale contradicts the data it ships with. When nothing meets the target, the solver falls back to the full candidate pool and correctly returns meets_target: false - yet the wording still claims the stack 'still holds the product for its target shelf life'. Measured: Potato Chips 35% fat / 180 days at 65% RH reports meets_target = false on all three tiers (max barrier reaches 147.2 of 180 days).
- Physics never validated against reality. No measured OTR/MVTR or shelf-life trials to compare against, and no light-barrier, headspace-oxygen, pasteurisation or seal-integrity term - which is why dairy reads optimistically (Pasteurized Milk at a 14-day target is offered a paperboard/LDPE pouch predicted at 30 days).
- No real-time transport at all. No WebSocket, no SSE. Every call is blocking, with a 12 s client timeout and a 60 s LLM timeout, so an LLM chat turn is a 5-30 s blank wait.

3. What kind of database this project needs

Eight logical collections are used today through a small DocumentStore interface (put / get / update / list / delete, with filters, order_by and limit). That separation is the good news: a Postgres implementation of the same interface drops in without touching a single router.

Collection	One record holds	How it is queried	Size
products	One food spec per project (~20 fields)	get by id; list filtered by userId, ordered createdAt	~1 KB
simulations	Run params + resultTimeSeries (180 points x several series)	put per run; get by id; list filtered by productId	20-60 KB
recommendations	3 tiers, metrics, why[] reasoning	put on generate; list filtered by productId	~5 KB
audits	Vision verdict + layer analysis + image ref	put; list filtered by productId	2-10 KB + image
chat_messages	One row per turn: sessionId, role, content	put per turn; list filtered by sessionId	~1 KB
suppliers	Catalogue entry: grade, region, price/kg, rating	list ordered by rating; get by id	~1 KB
rfq_requests	Outbound quote request to a supplier	put; list	~1 KB
project_summary	Roll-up per project	get / list	~2 KB

What the query load actually looks like: equality filters on userId / productId / sessionId, ordering by createdAt and by rating, limit-offset paging, and a write chain (product -> simulation -> recommendation) that should be transactional. The spec fields also evolve, which argues against a rigid

column-per-field table.

Recommended engine: PostgreSQL 16, managed (Neon, Supabase or RDS). Relational tables for what is filtered and joined (products, suppliers, rfq_requests, chat_messages); JSONB for shapes that are still moving (the physics time series and per-tier payloads); foreign keys with ON DELETE CASCADE from simulations, recommendations and audits to products. Indexes: (userId, createdAt DESC) on products; (productId, createdAt DESC) on simulations / recommendations / audits; (sessionId, createdAt) on chat_messages; (rating DESC) on suppliers. Connection pooling is required, not optional - the API is async FastAPI.

Implied schema

```
products(id pk, user_id, name, category, spec jsonb, created_at)
simulations(id pk, product_id fk, kind, params jsonb, series jsonb, created_at)
recommendations(id pk, product_id fk, tiers jsonb, used_llm bool, created_at)
audits(id pk, product_id fk, verdict jsonb, image_uri text, created_at)
chat_messages(id pk, session_id, role, content, created_at)
suppliers(id pk, name, region, rating, meta jsonb)
rfq_requests(id pk, supplier_id fk, payload jsonb, status, created_at)
```

- **Cache (Redis).** The 1,120-candidate search costs 276 ms and is a pure function of (spec, storage conditions). Memoising it by spec hash turns repeat views into roughly 0 ms. Redis also holds rate-limit counters and job state.
- **Object storage (S3-compatible).** Audit images and exported report PDFs belong in a bucket with a URL stored in the row, never base64 inside Postgres. reportlab already renders the PDF export server-side.
- **Migrations, backups and retention.** Alembic for schema versioning (there is none today); daily snapshots with point-in-time recovery, 7-30 day retention; and a written retention/deletion policy, because chat transcripts and audits hold PII and product specs can be commercially sensitive.
- **Sizing, and why not Firestore.** One project (product + 3 simulations + a recommendation + 1-2 audits) is roughly 100-200 KB excluding images, so 10,000 projects a year is 1-2 GB - a single small managed instance covers years. A FirestoreStore already exists and handles the simple get/put paths, but the filtered-and-ordered list calls are exactly where NoSQL charges a composite index per filter combination; keep it only if you specifically want mobile offline sync.

4. What kind of real-time API this app needs

The API today is pure request/response REST: no WebSocket, no SSE, no streaming endpoint. The client timeout is 12 s, the LLM timeout is configured at 60 s, and image upload is one blocking multipart POST.

Interaction	Today	What it actually needs
Chat reply	5-30 s blank wait for one JSON blob (or instant canned text)	SSE stream of tokens (text/event-stream), first token under 1 s. Highest-value real-time win.
Audit image analysis	One blocking POST, no progress, no cancel	202 Accepted + jobId, then SSE or poll for the verdict; upload progress on the client.
Simulation runs	15-20 ms, synchronous	Nothing. Keep it synchronous.
Recommendation search	276 ms, synchronous	Nothing. Cache the result in Redis.
What-if sweeps (multi-variant)	Not implemented	Background job queue with progress events.
Dashboard / History	Re-fetched when the screen opens	Fine as-is; optional ETag / conditional GET.
Job notifications	None	FCM push when a background job finishes.

Choose SSE over WebSocket. The traffic is one-directional (server to client progress and tokens), SSE runs over plain HTTP/1.1, reconnects automatically, and survives proxies and flaky mobile networks far better than a socket. WebSocket only becomes worth it if collaborative multi-user editing is ever built, which nothing in this app needs today.

Also required before this counts as a real API: URL versioning (/v1); JWT auth with refresh tokens; per-user rate limits; idempotency keys on POST /products and /simulate so a retry cannot duplicate a project; request-size limits on uploads; gzip; structured logs with request tracing; and health/readiness probes (a /health endpoint already exists and reports storage status).

5. Roadmap from demo to production

Priority	Work item	Effort
P0 - ship blocker	Deploy the FastAPI service (Docker + Fly/Render/Cloud Run) and rebuild the APK with --dart-define=API_BASE_URL=https://... . Without this the app is a demo, not a product.	1 day
P0 - ship blocker	Implement the Postgres DocumentStore and switch STORAGE_BACKEND off memory, so data survives restarts.	1-2 days
P0 - ship blocker	Generate a real release keystore and replace the debug signing config.	Half a day
P0 - ship blocker	Fix the tier rationale text so it can never claim a target the stack missed.	Hours
P1 - usable by real users	Authentication (JWT), per-user rate limiting and data isolation.	1-2 days
P1 - usable by real users	SSE streaming for the chat assistant, and object storage for audit images and report exports.	2 days
P1 - usable by real users	Rebuild and commit the APK so the shipped binary matches main.	Half a day
P2 - trustworthy results	Validate the physics against measured OTR/MVTR and real shelf-life trials; calibrate or add the missing light, headspace-oxygen and seal-integrity terms.	Weeks
P2 - trustworthy results	Background job queue with FCM notifications, plus monitoring, error tracking and an audit trail for spec changes.	4-6 days
P2 - trustworthy results	Offline-first local cache on the device so the wizard keeps working with no signal.	1 week

Bottom line: the 3D mockup studio and the physics/solver backend are genuinely real and worth keeping. What stands between this and a product is not more features - it is deployment, a persistent database, authentication and release signing, followed by honest validation of the physics against measured data.